

Examen 31 de mayo de 2021

Bootstrapping

- Descripción: Consiste en un proceso con el cual podremos a través de la combinación de compiladores más “simples” y “sencillos” obtener compiladores más complicados de implementar. A la hora de llevar acabo esto debemos de tener en cuenta que los compiladores se componen de 3 lenguajes:
 - Lenguaje Fuente (F): Lenguaje del fichero fuente a utilizar.
 - Lenguaje de Implementación (I): Lenguaje con el cual esta escrito el compilador.
 - Lenguaje Objeto (O): Lenguaje del fichero obtenido tras aplicar la compilación.
- A la hora de utilizar el bootstrapping, lo que se busca es a través de combinaciones de compiladores obtener más compiladores. Una expresión general para entender este funcionamiento sería: $F_I O + I_M N$ generando así el $F_N O$
- Ejemplos
 - Algunos ejemplos concretos serían en el caso de que $F = I$ significaría que nos encontramos con un autocompilador, en este caso este compilador deberá de ser compilado para poder hacer usado.
 - $I \neq 0$ Si ocurre esto, estamos hablando de compilador cruzado, en este caso este compiladores generará código para otra máquina.

Análisis léxico: Tipos de reconocimiento de las palabras claves

- Implícito: En este caso las palabras claves se encuentran preescritas en la tabla de símbolos, en este caso su reconocimiento es más lento, pero simplifica mucho más la implementación del analizador léxico.
- Explícito: En este caso se definen reglas para reconocer cada una de las palabras reservadas del lenguaje, minimizando así el tiempo para reconocer cada una de las palabras. El problema reside en la complejidad y que dificulta más la implementación del analizador.

Componentes léxicos

- Dada la siguiente expresión regular: $(a + b) c^* (a+b)$

1. Utiliza el **Algoritmo de Thompson** para construir un AFN equivalente a la expresión regular.
2. Utiliza el **Algoritmo de Construcción de Subconjuntos** para obtener un AFD equivalente al AFN obtenido en el apartado anterior.
3. **Minimiza**, si es posible, el AFD obtenido en el apartado anterior.
4. Comprueba si el último autómata obtenido reconoce la sentencia: **a c c c b**

Resolución:

Como podemos ver en la **Figura 1**, se pueden apreciar los diferentes elementos de la expresión regular. En la **Figura 2**. Una vez realizado esto debemos de aplicar el **algoritmo de Construcción de Subconjuntos**, para obtener el AFD equivalente.

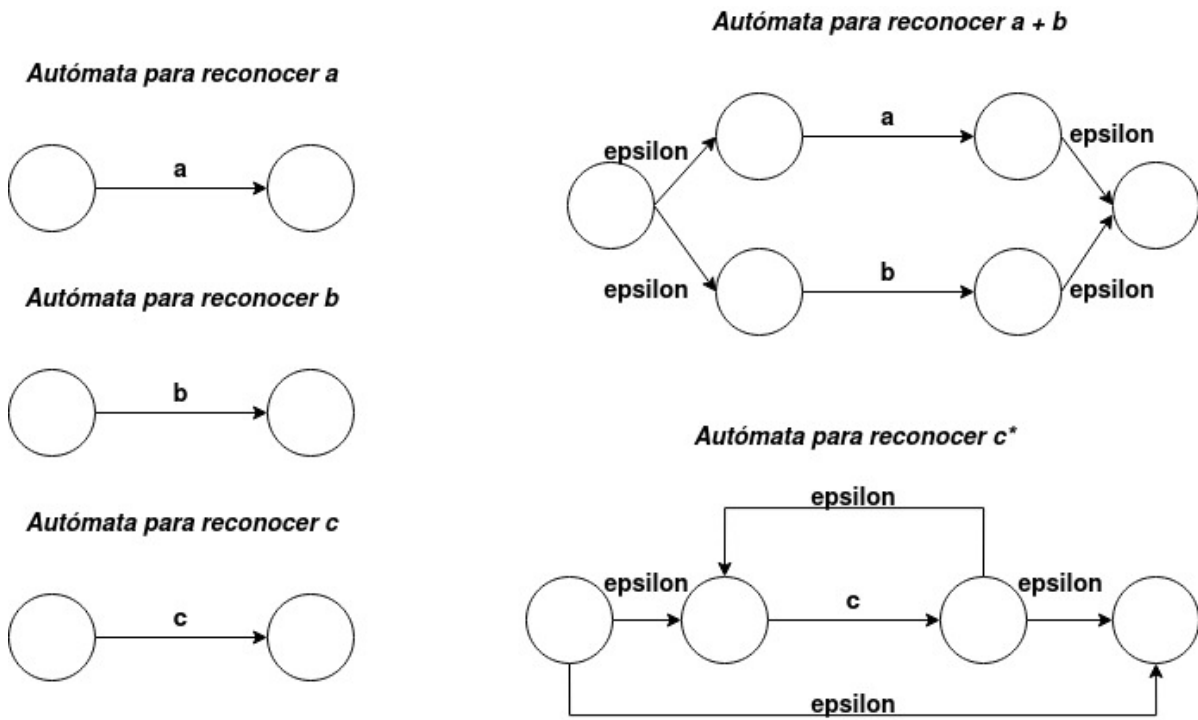


Figura 1: Autómatas Para Reconocer partes del Autómata Finito No Determinista

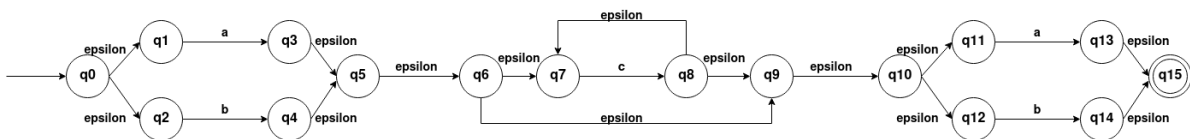


Figura 2: Autómatas Finito No Determinista - Final

$$p_0 = \text{clausura} - \epsilon(q_0) = \{q_0, q_1, q_2\} \quad (1)$$

Ahora debemos de indicar las diferentes transiciones de p_0 (2)

$$\delta_D(p_0, a) = \text{clausura} - \epsilon\left(\bigcup_{q' \in p_0} \delta_N(q', a)\right) = \text{clausura} - \epsilon(\delta_N(q_0, a) \cup \delta_N(q_1, a) \cup \delta_N(q_2, a)) = \quad (3)$$

$$\text{clausura} - \epsilon(\{q_3\}) = \{q_3, q_5, q_6, q_7, q_9, q_{10}, q_{11}, q_{12}\} = p_1 \quad (4)$$

(5)

$$\delta_D(p_0, b) = \text{clausura} - \epsilon(\bigcup_{q' \in p_0} \delta_N(q', b)) = \text{clausura} - \epsilon(\delta_N(q_0, b) \cup \delta_N(q_1, b) \cup \delta_N(q_2, b)) = \quad (6)$$

$$\text{clausura} - \epsilon(\{q_3\}) = \{q_4, q_5, q_6, q_7, q_9, q_{10}, q_{11}, q_{12}\} = p_2 \quad (7)$$

Ahora debemos de indicar las diferentes transiciones de p_1 (8)

$$\delta_D(p_1, a) = \text{clausura} - \epsilon(\bigcup_{q' \in p_1} \delta_N(q', a)) = \text{clausura} - \epsilon(\delta_N(q_3, a) \cup \delta_N(q_5, a) \cup \delta_N(q_6, a) \quad (9)$$

$$\cup \delta_N(q_7, a) \cup \delta_N(q_9, a) \cup \delta_N(q_{10}, a) \cup \delta_N(q_{11}, a) \cup \delta_N(q_{12}, a)) = \text{clausura} - \epsilon(\{q_{13}\}) \quad (10)$$

$$= \{q_{13}, q_{15}\} = p_3 \quad (11)$$

$$\delta_D(p_1, b) = \text{clausura} - \epsilon(\bigcup_{q' \in p_1} \delta_N(q', b)) = \text{clausura} - \epsilon(\delta_N(q_3, b) \cup \delta_N(q_5, b) \cup \delta_N(q_6, b) \quad (12)$$

$$\cup \delta_N(q_7, b) \cup \delta_N(q_9, b) \cup \delta_N(q_{10}, b) \cup \delta_N(q_{11}, b) \cup \delta_N(q_{12}, b)) = \text{clausura} - \epsilon(\{q_{14}\}) \quad (13)$$

$$= \{q_{14}, q_{15}\} = p_4 \quad (14)$$

$$\delta_D(p_1, c) = \text{clausura} - \epsilon(\bigcup_{q' \in p_1} \delta_N(q', c)) = \text{clausura} - \epsilon(\delta_N(q_3, c) \cup \delta_N(q_5, c) \cup \delta_N(q_6, c) \quad (15)$$

$$\cup \delta_N(q_7, c) \cup \delta_N(q_9, c) \cup \delta_N(q_{10}, c) \cup \delta_N(q_{11}, c) \cup \delta_N(q_{12}, c)) = \text{clausura} - \epsilon(\{q_8\}) \quad (16)$$

$$= \{q_7, q_8, q_9, q_{10}, q_{11}, q_{12}\} = p_5 \quad (17)$$

Ahora debemos de indicar las diferentes transiciones de p_2 (18)

$$\delta_D(p_2, a) = \text{clausura} - \epsilon(\bigcup_{q' \in p_2} \delta_N(q', a)) = \text{clausura} - \epsilon(\delta_N(q_4, a) \cup \delta_N(q_5, a) \cup \delta_N(q_6, a) \quad (19)$$

$$\cup \delta_N(q_7, a) \cup \delta_N(q_9, a) \cup \delta_N(q_{10}, a) \cup \delta_N(q_{11}, a) \cup \delta_N(q_{12}, a)) = \text{clausura} - \epsilon(\{q_{13}\}) \quad (20)$$

$$= \{q_{13}, q_{15}\} = p_3 \quad (21)$$

$$\delta_D(p_2, b) = \text{clausura} - \epsilon(\bigcup_{q' \in p_2} \delta_N(q', b)) = \text{clausura} - \epsilon(\delta_N(q_4, b) \cup \delta_N(q_5, b) \cup \delta_N(q_6, b) \quad (22)$$

$$\cup \delta_N(q_7, b) \cup \delta_N(q_9, b) \cup \delta_N(q_{10}, b) \cup \delta_N(q_{11}, b) \cup \delta_N(q_{12}, b)) = \text{clausura} - \epsilon(\{q_{14}\}) \quad (23)$$

$$= \{q_{14}, q_{15}\} = p_4 \quad (24)$$

$$\delta_D(p_2, c) = \text{clausura} - \epsilon(\bigcup_{q' \in p_2} \delta_N(q', c)) = \text{clausura} - \epsilon(\delta_N(q_4, c) \cup \delta_N(q_5, c) \cup \delta_N(q_6, c) \quad (25)$$

$$\cup \delta_N(q_7, c) \cup \delta_N(q_9, c) \cup \delta_N(q_{10}, c) \cup \delta_N(q_{11}, c) \cup \delta_N(q_{12}, c)) = \text{clausura} - \epsilon(\{q_8\}) \quad (26)$$

$$= \{q_7, q_8, q_9, q_{10}, q_{11}, q_{12}\} = p_5 \quad (27)$$

Ahora debemos de indicar las diferentes transiciones de p_3 , No posee transiciones (28)

Ahora debemos de indicar las diferentes transiciones de p_4 , No posee transiciones (29)

Ahora debemos de indicar las diferentes transiciones de p_5 (30)

$$\delta_D(p_5, a) = \text{clausura} - \epsilon(\bigcup_{q' \in p_5} \delta_N(q', a)) = \text{clausura} - \epsilon(\delta_N(q_7, a) \cup \delta_N(q_8, a) \quad (31)$$

$$\cup \delta_N(q_9, a) \cup \delta_N(q_{10}, a) \cup \delta_N(q_{11}, a) \cup \delta_N(q_{12}, a)) = \text{clausura} - \epsilon(\{q_{13}\}) \quad (32)$$

$$= \{q_{13}, q_{15}\} = p_3 \quad (33)$$

$$\delta_D(p_5, b) = \text{clausura} - \epsilon(\bigcup_{q' \in p_5} \delta_N(q', b)) = \text{clausura} - \epsilon(\delta_N(q_7, b) \cup \delta_N(q_8, b) \quad (34)$$

$$\cup \delta_N(q_9, b) \cup \delta_N(q_{10}, b) \cup \delta_N(q_{11}, b) \cup \delta_N(q_{12}, b)) = \text{clausura} - \epsilon(\{q_{14}\}) \quad (35)$$

$$= \{q_{14}, q_{15}\} = p_4 \quad (36)$$

$$\delta_D(p_5, c) = \text{clausura} - \epsilon(\bigcup_{q' \in p_5} \delta_N(q', c)) = \text{clausura} - \epsilon(\delta_N(q_7, c) \cup \delta_N(q_8, c) \quad (37)$$

$$\cup \delta_N(q_9, c) \cup \delta_N(q_{10}, c) \cup \delta_N(q_{11}, c) \cup \delta_N(q_{12}, c)) = \text{clausura} - \epsilon(\{q_8\}) = p_5 \quad (38)$$

El autómata obtenido de la construcción de subconjuntos sería el que podemos ver en la **Figura 3**.

Una vez realizado esto deberemos de simplificar el autómata.

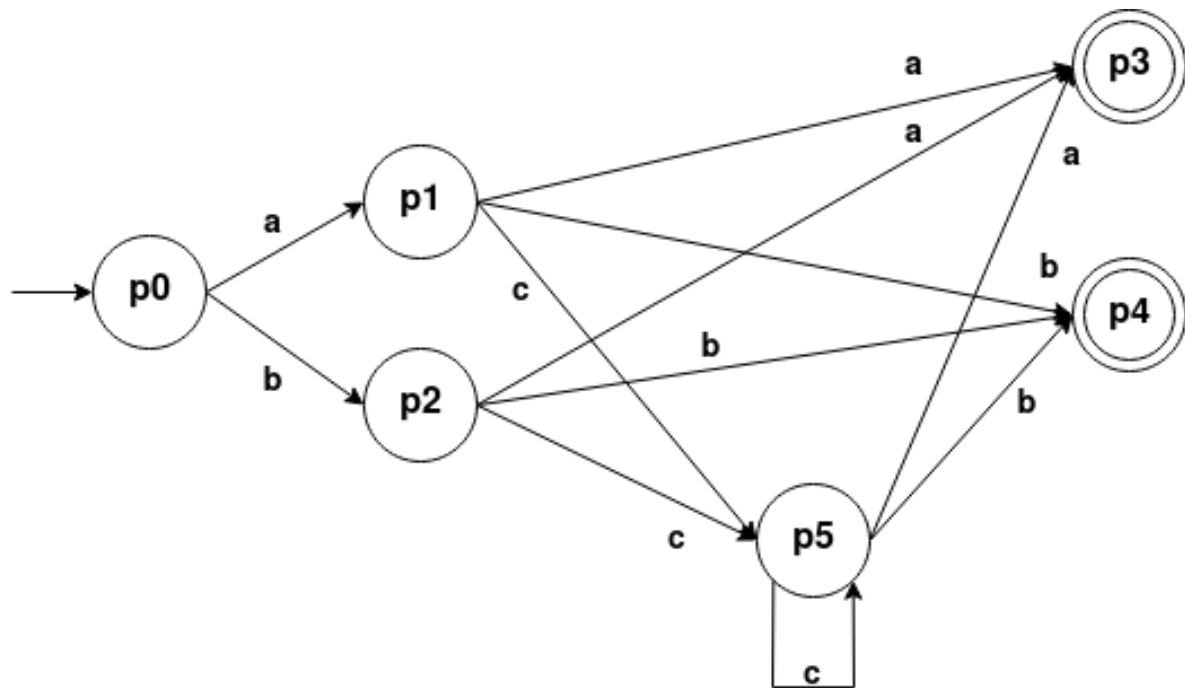


Figura 3: Autómata Finito Determinista

Minimización de estados:

Para ello en primer lugar agruparemos todos los estados finales en un solo estado y los no finales en otro estado.

- **Estados No Finales:** $a_0 \rightarrow \{p_0, p_1, p_2, p_5\}$
- **Estados Finales:** $a_1 \rightarrow \{p_3, p_4\}$

En primer lugar agregaremos todos los estados a $NUEVO = \{a_0, a_1\}$, marcamos a_0 , y comprobamos si estos estados son "homogéneos". Como podemos apreciar en la **Tabla 1**, podemos apreciar como los estados p_0 y p_1, p_2, p_5 no son homogéneos, siendo necesaria su división en dos estados. De forma que los estados actuales serían:

- $a_0 \rightarrow \{p_0\}$
- $a_1 \rightarrow \{p_3, p_4\}$
- $a_2 \rightarrow \{p_1, p_2, p_5\}$

δ	a	b	c
p_0	a_0	a_0	-
p_1	a_1	a_1	a_0
p_2	a_1	a_1	a_0
p_3	-	-	-
p_4	-	-	-
p_5	a_1	a_1	a_0

Cuadro 1: Transiciones Paso 1 de Minimización

En este caso las transiciones cambiarían a las que podemos ver en la **Tabla 2**, en este caso podemos apreciar como los estados que componen el estado a_1 son iguales, lo cual ocurre también

en el estado a_2 . En el caso del estado a_0 , podemos ver que al componerse de uno solo no se puede dividir más.

δ	a	b	c
p_0	a_2	a_2	-
p_1	a_1	a_1	a_2
p_2	a_1	a_1	a_2
p_3	-	-	-
p_4	-	-	-
p_5	a_1	a_1	a_2

Cuadro 2: Transiciones Paso 2 de Minimización

Siendo el **autómata final** el que podemos ver en la **Figura 4**. Una vez realizado esto deberemos de comprobar la sentencia **a c c c b**.

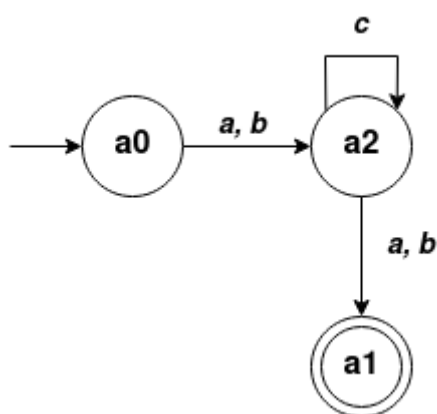


Figura 4: Autómata Finito Determinista

$$\delta(a_0, acccb) \vdash \delta(a_2, cc cb) \quad (39)$$

$$\vdash \delta(a_2, cc b) \quad (40)$$

$$\vdash \delta(a_2, cb) \quad (41)$$

$$\vdash \delta(a_2, b) \quad (42)$$

$$\vdash \delta(a_1, \epsilon) \quad (43)$$

Análisis descendente predictivo

- Considérese la siguiente gramática de contexto libre:

P = {
 1) $S \rightarrow S D$
 2) $S \rightarrow \epsilon$
 3) $D \rightarrow \text{id} : \text{array} [\text{entero} \dots \text{entero}] \text{ of } T ;$
 4) $T \rightarrow \text{integer}$
 5) $T \rightarrow \text{real}$
 }

- Esta gramática permite generar declaraciones de variables de tipo **array** en el lenguaje Pascal. Ejemplos:

- `datos : array [1 .. 4] of integer`
- `valor : array [-10 .. 30] of real`

- donde: `datos` y `valor` son **identificadores**

1. Elimina la recursividad por la izquierda y factoriza la gramática por la izquierda.

2. A partir de la gramática obtenida en el apartado “a”:

- Construye los conjuntos “primero” y “siguiente” de los símbolos no terminales.
- Construye la tabla de análisis descendente predictivo.
- Utiliza el método de recuperación de errores de “nivel de frase” para completar la tabla predictiva.
- Utiliza la tabla predictiva para realizar un análisis descendente no recursivo de la siguiente declaración errónea: **datos : [1 .. 4 of integer real**

Resolución: En primer lugar deberemos de eliminar la recursividad de la izquierda y después factorizaremos, siendo como resultado las reglas que podemos ver a continuación:

- 1) $S \rightarrow D S'$
- 2) $S' \rightarrow D$
- 3) $S' \rightarrow \epsilon$
- 4) $D \rightarrow \text{id} : \text{array} [\text{entero} \dots \text{entero}] \text{ of } T ;$
- 5) $T \rightarrow \text{integer}$
- 6) $T \rightarrow \text{real}$

Una vez tenemos la gramática sin recursividad inmediata por la izquierda y factorizada, podemos construir el conjunto “primero” y “siguiente”.

- Regla 1) $S \rightarrow D S'$
- Primero (D) $\subseteq$ Primero(S)
- Regla 2) $S' \rightarrow D$
- Primero (D) $\subseteq$ Primero(S')
- Regla 3) $S' \rightarrow \epsilon$
- $\epsilon \in$ Primero(S')
- Regla 4) $D \rightarrow \text{id} : \text{array} [\text{entero} \dots \text{entero}] \text{ of } T ;$
- `id` $\in$ Primero(D)
- Regla 5) $T \rightarrow \text{integer}$
- `integer` $\in$ Primero(T)
- Regla 6) $T \rightarrow \text{real}$
- `real` $\in$ Primero(T)

Estado	Primero
S	id
S'	id, ϵ
D	id
T	entero, real

Cuadro 3: Conjunto Primero de la gramática obtenida

Una vez construido el conjunto primero deberemos de proceder con el **conjunto siguiente**:

- Regla 1) $S \rightarrow D S'$
 - $\$ \in \text{Siguiente}(S)$
 - $\text{Siguiente}(S) \subseteq \text{Siguiente}(S')$
 - $\text{Primero}(S') - \{\epsilon\} \subseteq \text{Siguiente}(D)$
 - Como $\epsilon \in \text{Primero}(S') \rightarrow \text{Siguiente}(S) \subseteq \text{Siguiente}(D)$
- Regla 2) $S' \rightarrow D$
 - $\text{Siguiente}(S') \subseteq \text{Siguiente}(D)$
- Regla 4) $D \rightarrow \text{id} : \text{array} [\text{entero} \dots \text{entero}] \text{ of } T ;$
 - $; \in \text{Siguiente}(T)$

Estado	Primero	Siguiente
S	id	\$
S'	id, ϵ	\$
D	id	\$, id
T	entero, real	;

Cuadro 4: Conjunto Primero y Siguiente de la gramática obtenida

Una vez tenemos construido el conjunto siguiente podemos proceder a construir la tabla **predictiva**. Para ello deberemos de definir las diferentes transiciones posibles.

- Regla 1) $S \rightarrow D S'$
 - Como $\text{Primero}(D S') - \{\epsilon\} = \{\text{id}\}$
 - $M[S, \text{id}] = 1$
- Regla 2) $S' \rightarrow D$
 - Como $\text{Primero}(D) - \{\epsilon\} = \{\text{id}\}$
 - $M[S', \text{id}] = 2$
- Regla 3) $S' \rightarrow \epsilon$
 - Como $\text{Primero}(\epsilon) - \{\epsilon\} = \emptyset$, entonces debemos de utilizar el Siguiente (S') = $\{\$\}$
 - $M[S', \$] = 3$
- Regla 4) $D \rightarrow \text{id} : \text{array} [\text{entero} \dots \text{entero}] \text{ of } T ;$
 - Como $\text{Primero}(\alpha) - \{\epsilon\} = \text{id}$
 - $M[D, \text{id}] = 4$
- Regla 5) $T \rightarrow \text{integer}$
 - Como $\text{Primero}(\text{integer}) - \{\epsilon\} = \text{integer}$
 - $M[D, \text{integer}] = 5$

- Regla 6) $T \rightarrow \text{real}$
 - Como Primero(real) - $\{\epsilon\} = \text{real}$
 - $M[D, \text{real}] = 6$

Estado	id	:	array	[	entero	..	]	of	;	integer	real	\$
S	1											
S'	2											3
D	4											
T										5	6	

Cuadro 5: Tabla Predictiva

Una vez definida la **Tabla predictiva**, debemos de definir ahora las diferentes funciones para tratar el error, además de ampliar la tabla.

- **E1** Falta id, insertar id en la entrada.
- **E2** Falta :, insertar : en la entrada.
- **E3** Falta array, insertar array en la entrada.
- **E4** Falta [, insertar [en la entrada.
- **E5** Falta entero, insertar entero en la entrada.
- **E6** Falta .., insertar .. en la entrada.
- **E7** Falta], insertar] en la entrada.
- **E8** Falta of, insertar of en la entrada.
- **E9** Falta tipo, insertar integer en la entrada.
- **E10** Falta ;, insertar ; en la entrada.
- **E11** Símbolos inesperado, borrar símbolo de la entrada
- **E12** Final inesperado

Estado	id	:	array	[	entero	..	]	of	;	integer	real	\$
S	1	E1	E11	E11	E11	E11	E11	E11	E11	E11	E11	E12
S'	2	E1	E11	E11	E11	E11	E11	E11	E11	E11	E11	3
D	4	E1	E11	E11	E11	E11	E11	E11	E11	E11	E11	E12
T	E11	E11	E11	E11	E11	E11	E11	E11	E9	5	6	E12

id	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E12
:	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E12
array	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E12
[	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E12
entero	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E12
..	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E12
]	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E12
of	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E12
;	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E10
integer										E11	E11	E12
real											E11	E12
\$	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	E11	Aceptar

Cuadro 6: Tabla Predictiva Ampliada

Ahora procederemos a reconocer la sentencia **datos** : [1 .. 4 of integer real

Pila	Entrada	Acción
\$ S	id : [ent .. ent of integer real \$	S → D S'
\$ S' D	id : [ent .. ent of integer real \$	S → id : array [entero .. entero] of T ;
\$ S' ; T of] entero .. entero [array : id	id : [ent .. ent of integer real \$	Emparejar
\$ S' ; T of] entero .. entero [array :	: [ent .. ent of integer real \$	Emparejar
\$ S' ; T of] entero .. entero [array	[ent .. ent of integer real \$	E3, falta array
\$ S' ; T of] entero .. entero [array	array [ent .. ent of integer real \$	Emparejar
\$ S' ; T of] entero .. entero [	[ent .. ent of integer real \$	Emparejar
\$ S' ; T of] entero .. entero	ent .. ent of integer real \$	Emparejar
\$ S' ; T of] entero ..	.. ent of integer real \$	Emparejar
\$ S' ; T of] entero	ent of integer real \$	Emparejar
\$ S' ; T of]	of integer real \$	E7, falta]
\$ S' ; T of]	] of integer real \$	Emparejar
\$ S' ; T of	of integer real \$	Emparejar
\$ S' ; T	integer real \$	Regla 5) T → integer
\$ S' ; integer	integer real \$	Emparejar
\$ S' ;	real \$	E11 , Simbolo inesperado
\$ S' ;	\$	E10, Falta ;
\$ S' ;	;	Emparejar
\$ S'	\$	Regla 3) S' → ε
\$	\$	Aceptar

Cuadro 7: ADP - Análisis sentencia

Análisis sintáctico ascendente LR (1) - canónico

- Considérese la siguiente gramática de contexto libre.

P = {
 1) $S \rightarrow S D$
 2) $S \rightarrow \epsilon$
 3) $D \rightarrow \text{type identificador} = \text{set of } T$;
 4) $T \rightarrow \text{char}$
 5) $T \rightarrow \text{real}$
 }

- Esta gramática permite generar declaraciones de variables de tipo conjunto (set) en el lenguaje Pascal. Ejemplos:

- type letras = set of char
- type datos = set of integer

- donde: letras y datos son **identificadores**

1. Construye la colección canónica de LR(1)-elementos del análisis LR-canónico.
2. Dibuja el autómata que reconoce los prefijos viables.
3. Construye la tabla de análisis sintáctico LR-canónico: partes acción e ir_a.
4. Construye la tabla de análisis sintáctico SLR: partes acción e ir_a
5. Utiliza el método recuperación de errores de “**nivel de frase**” para completar la tabla de análisis LR-canónico.
6. Utiliza la tabla SLR para analizar la siguiente declaración errónea: **type type datos = of char integer;**

Resolución:

En primer lugar deberemos de ampliar la gramática:

P = {
 1') $S' \rightarrow S$
 1) $S \rightarrow S D$
 2) $S \rightarrow \epsilon$
 3) $D \rightarrow \text{type identificador} = \text{set of } T$;
 4) $T \rightarrow \text{char}$
 5) $T \rightarrow \text{real}$
 }

Ahora una vez ampliada la gramática, empezaremos a construir la colección canónica de LR(1).

- $$I_0 = \text{clausura}(\{[S' \rightarrow \cdot S, \$]\}) = \{[S' \rightarrow \cdot S, \$], [S \rightarrow \cdot SD, \$, type], [S \rightarrow \cdot, \$, type]\} \quad (44)$$
- Transiciones de I_0 (45)
- $$Ira(I_0, S) = \text{clausura}(\{[S' \rightarrow S \cdot, \$], [S \rightarrow S \cdot D, \$, type]\}) = \quad (46)$$
- $$= \{[S' \rightarrow S \cdot, \$], [S \rightarrow S \cdot D, \$, type], [D \rightarrow \cdot \text{type identificador} = \text{set of } T ;, \$, type]\} = I_1 \quad (47)$$
- Transiciones de I_1 (48)
- $$Ira(I_1, D) = \text{clausura}(\{[S \rightarrow SD \cdot, \$, type]\}) = I_2 \quad (49)$$
- $$Ira(I_1, type) = \text{clausura}(\{[D \rightarrow \text{type} \cdot \text{identificador} = \text{set of } T ;, \$, type]\}) = I_3 \quad (50)$$
- Transiciones de I_2 , No posee (51)
- Transiciones de I_3 (52)
- $$Ira(I_3, \text{identificador}) = \text{clausura}(\{[D \rightarrow \text{type identificador} \cdot = \text{set of } T ;, \$, type]\}) = I_4 \quad (53)$$
- Transiciones de I_4 (54)
- $$Ira(I_4, =) = \text{clausura}(\{[D \rightarrow \text{type identificador} = \cdot \text{set of } T ;, \$, type]\}) = I_5 \quad (55)$$
- Transiciones de I_5 (56)
- $$Ira(I_5, \text{set}) = \text{clausura}(\{[D \rightarrow \text{type identificador} = \text{set} \cdot \text{of } T ;, \$, type]\}) = I_6 \quad (57)$$
- Transiciones de I_6 (58)
- $$Ira(I_6, \text{of}) = \text{clausura}(\{[D \rightarrow \text{type identificador} = \text{set of} \cdot T ;, \$, type]\}) \quad (59)$$
- $$= \{[D \rightarrow \text{type identificador} = \text{set of} \cdot T ;, [T \rightarrow \cdot \text{char}, ;], [T \rightarrow \cdot \text{real}, ;]\} = I_7 \quad (60)$$
- Transiciones de I_7 (61)
- $$Ira(I_7, T) = \text{clausura}(\{[D \rightarrow \text{type identificador} = \text{set of } T \cdot ;, \$, type]\}) = \quad (62)$$
- $$\{[D \rightarrow \text{type identificador} = \text{set of } T \cdot ;, \$, type]\} = I_8 \quad (63)$$
- $$Ira(I_7, \text{char}) = \text{clausura}(\{[T \rightarrow \text{char} \cdot, ;]\}) = \quad (64)$$
- $$\{[T \rightarrow \text{char} \cdot, ;]\} = I_9 \quad (65)$$
- $$Ira(I_7, \text{real}) = \text{clausura}(\{[T \rightarrow \text{real} \cdot, ;]\}) = \quad (66)$$
- $$\{[T \rightarrow \text{real} \cdot, ;]\} = I_{10} \quad (67)$$
- Transiciones de I_8 (68)
- $$Ira(I_8, ;) = \text{clausura}(\{[D \rightarrow \text{type identificador} = \text{set of } T ; \cdot, \$, type]\}) = \quad (69)$$
- $$\{[D \rightarrow \text{type identificador} = \text{set of } T ; \cdot, \$, type]\} = I_{11} \quad (70)$$
- Transiciones de I_9 , No posee transiciones (71)
- Transiciones de I_{10} , No posee transiciones (72)
- Transiciones de I_{11} , No posee transiciones (73)

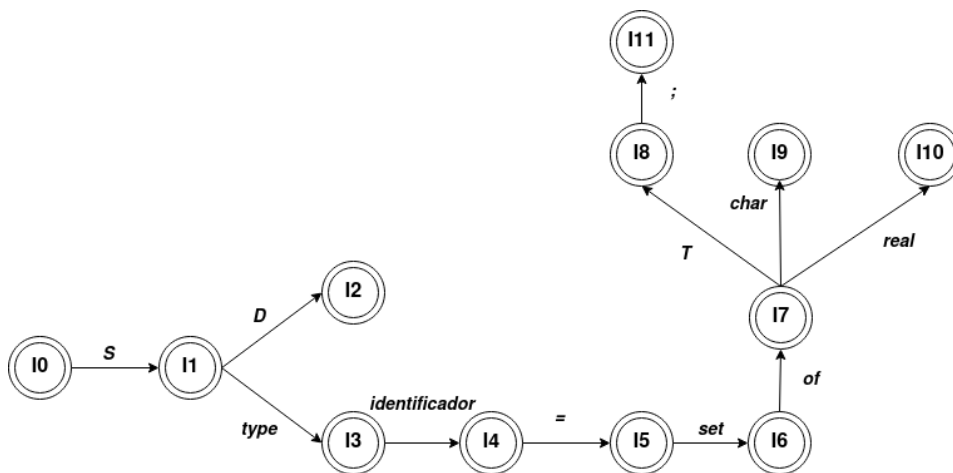


Figura 5: Autómata que reconoce los prefijos viables

Ahora deberemos de proceder a la construcción de la tabla.

Estado	type	id	=	set	of	;	char	real	\$	S	D	T
I_0	r2								r2	1		
I_1	d3								Aceptar		2	
I_2	r1								r1			
I_3		d4										
I_4			d5									
I_5				d6								
I_6					d7							
I_7							d9	d10				8
I_8						d11						
I_9						r4						
I_{10}						r5						
I_{11}	r3								r3			

Cuadro 8: Tabla de análisis sintáctico LR-canónico

Una vez construida la tabla “base” deberemos de agregar las diferentes funciones de recuperación de errores de **nivel de frase**.

- **E1** Falta type, insertar type en la entrada.
- **E2** Símbolo inesperado, borrar símbolo de la entrada.
- **E3** Falta id , insertar id en la entrada.
- **E4** Final inesperado, borrar símbolo de la pila.
- **E5** Falta tipo = insertar = en la entrada.
- **E6** Falta set , insertar set en la entrada.
- **E7** Falta of , insertar of en la entrada.
- **E8** Falta tipo , insertar int en la entrada.
- **E9** Falta “;”, insertar “;” en la entrada.

Estado	type	id	=	set	of	;	char	real	\$	S	D	T
I_0	r2	r2*	r2*	r2*	r2*	r2*	r2*	r2*	r2	1		
I_1	d3	E1	E2	E2	E2	E2	E2	E2	Aceptar		2	
I_2	r1	E1	E2	E2	E2	E2	E2	E2	r1			
I_3	E2	d4	E3	E2	E2	E2	E2	E2	E4			
I_4	E2	E2	d5	E5	E2	E2	E2	E2	E4			
I_5	E2	E2	E2	d6	E6	E2	E2	E2	E4			
I_6	E2	E2	E2	E2	d7	E2	E7	E7	E4			
I_7	E2	E2	E2	E2	E2	E8	d9	d10	E4			8
I_8	E9	E2	E2	E2	E2	d11	E2	E2	E9			
I_9	E2	E2	E2	E2	E2	r4	E2	E2	E4			
I_{10}	E2	E2	E2	E2	E2	r5	E2	E2	E4			
I_{11}	r3	E2	E2	E2	E2	E2	E2	E2	r3			

Cuadro 9: Tabla de análisis sintáctico LR-canónico con control de errores

Una vez finalizada la tabla, podemos proceder a analizar la sentencia: **type type datos = of char integer;**

Pila	Entrada	Acción
0	type type id = of char int; \$	r2
0 S 1	type type id = of char int; \$	d3
0 S 1 type 3	type id = of char int; \$	E2 Simbolo inesperado, borrar de la entrada
0 S 1 type 3	id = of char int; \$	d4
0 S 1 type 3 id 4	= of char int; \$	d5
0 S 1 type 3 id 4 = 5	of char int; \$	E6 Falta set, insertar en la entrada
0 S 1 type 3 id 4 = 5	set of char int; \$	d6
0 S 1 type 3 id 4 = 5 set 6	of char int; \$	d7
0 S 1 type 3 id 4 = 5 set 6 of 7	char int; \$	d9
0 S 1 type 3 id 4 = 5 set 6 of 7 char 9	int; \$	E2 Simbolo inesperado, borrar de la entrada
0 S 1 type 3 id 4 = 5 set 6 of 7 char 9	; \$	r4
0 S 1 type 3 id 4 = 5 set 6 of 7 T 8	; \$	d11
0 S 1 type 3 id 4 = 5 set 6 of 7 T 8 ; 11	\$	r3
0 S 1 D 2	\$	r1
0 S 1	\$	Aceptar

Cuadro 10: LR-canónico - Análisis sentencia

Diseño de una gramática de contexto libre

- Diseña una gramática que permita definir los atributos privados (private) de una clase de C++ como la siguiente

Ejemplo de Gramática

```
class Persona
{
    private:
        string _nombre;
        string _apellidos;
        int _edad;
        bool _sexo;
}
```

Un ejemplo de gramática podría ser:

1. $S \rightarrow D S$
2. $S \rightarrow \epsilon$
3. $D \rightarrow \text{class id } \{ \text{private: I } \}$
4. $I \rightarrow T \text{ id; I'}$
5. $I' \rightarrow T \text{ id; I'}$
6. $I' \rightarrow \epsilon$
7. $T \rightarrow \text{string}$
8. $T \rightarrow \text{int}$
9. $T \rightarrow \text{bool}$